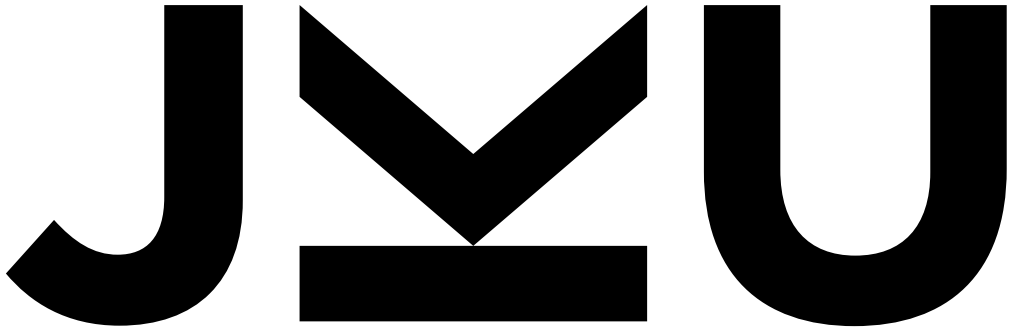
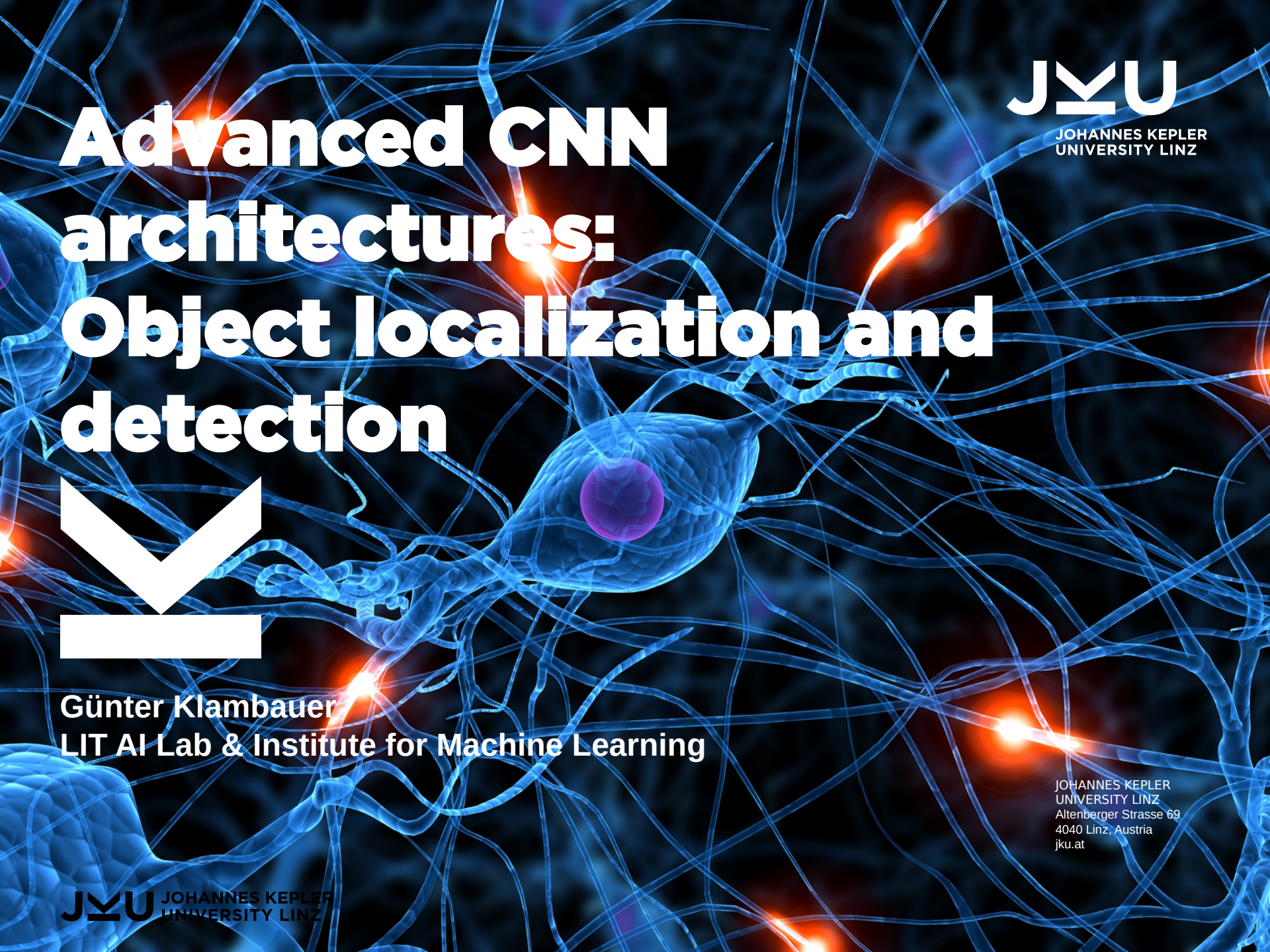


JOHANNES KEPLER
UNIVERSITY LINZ



Advanced CNN architectures: Object localization and detection

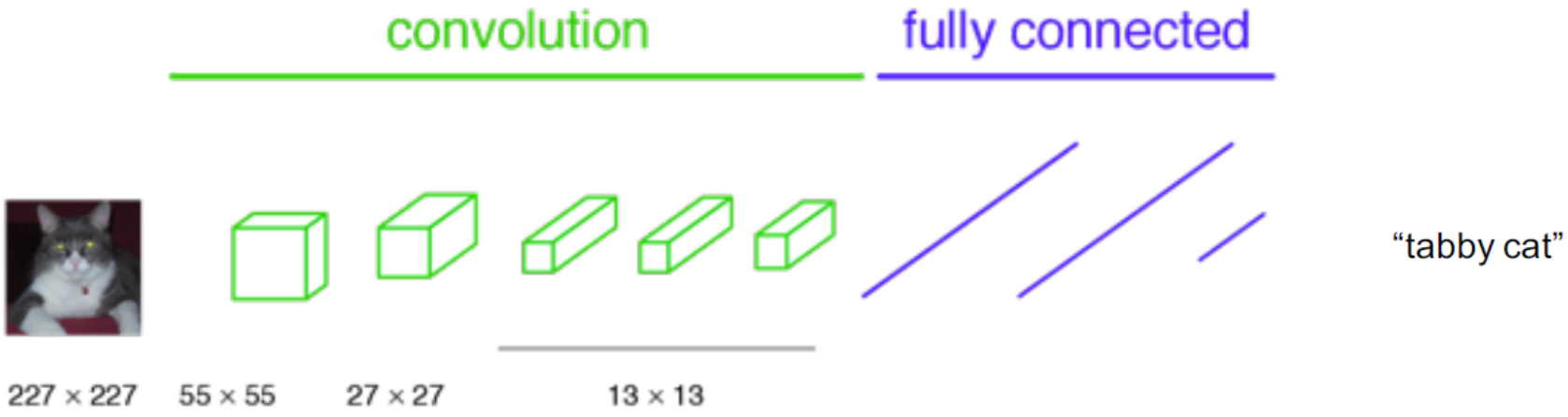
Günter Klambauer
LIT AI Lab & Institute for Machine Learning

JOHANNES KEPLER
UNIVERSITY LINZ
Altenberger Strasse 69
4040 Linz, Austria
jku.at

11. Advanced CNN architectures

- 1.1. Image Classification and Image Recognition
- 1.2. Semantic Segmentation
- **1.3. Object Detection and Localization**
- 1.4. Instance Segmentation

Recap: Fully Convolutional Network (FConvN)



Recap: IoU

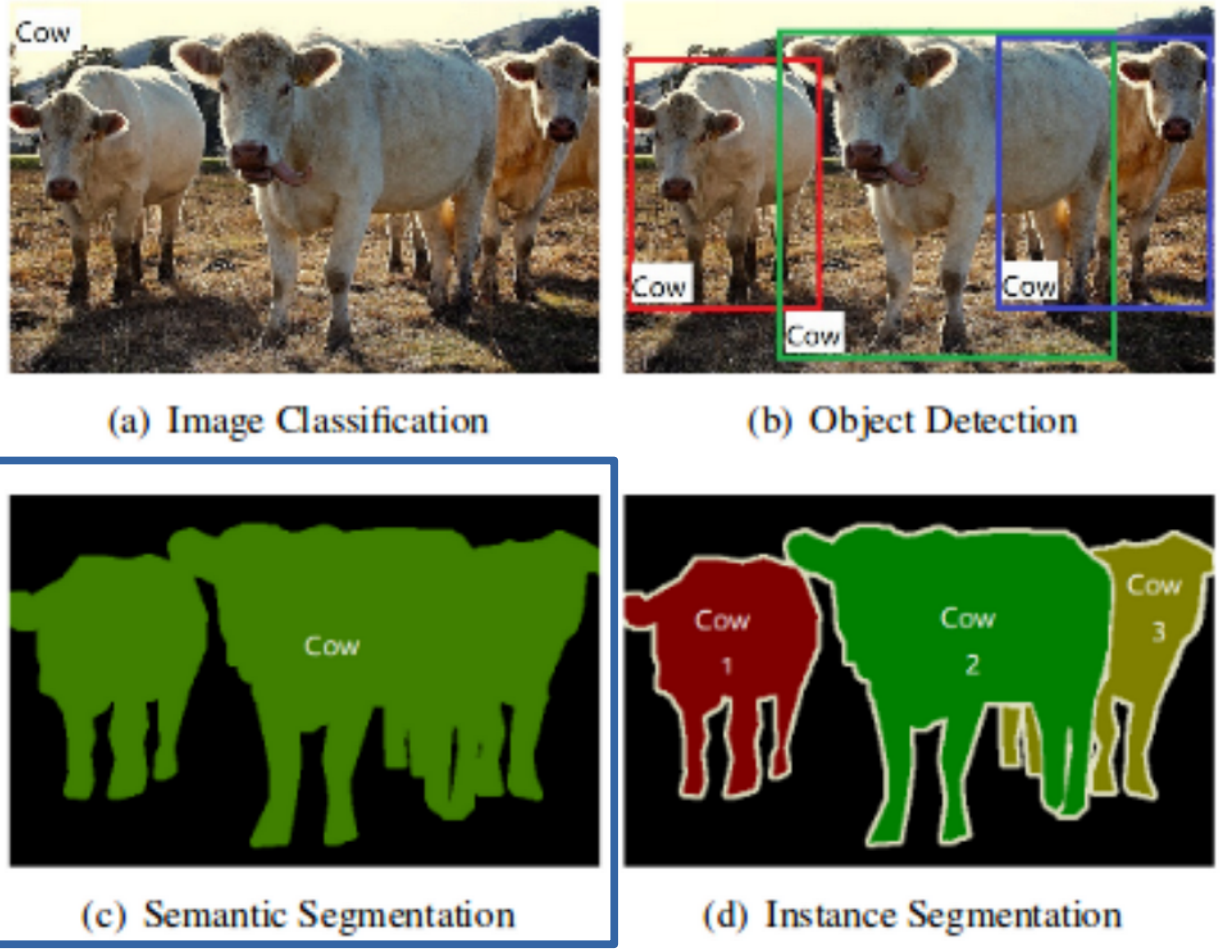
- Maximize the number of pixels that agree with the segmented image (label):
- *Intersection over Union* (IoU) or *Jaccard index* for two sets $\mathcal{A}, \mathcal{B}$ (of pixels)*:*

$$J(\mathcal{A}, \mathcal{B}) = \frac{|\mathcal{A} \cap \mathcal{B}|}{|\mathcal{A} \cup \mathcal{B}|} = \frac{|\mathcal{A} \cap \mathcal{B}|}{|\mathcal{A}| + |\mathcal{B}| - |\mathcal{A} \cap \mathcal{B}|}$$

Recap Semantic Segmentation

- 11.2.1 Task definition
- 11.2.2 Transposed convolutions
- 11.2.3 Dilated convolutions
- 11.2.4 Fully convolutional networks
- …
- 11.2.12 Auto DeepLab (2019)

Fundamental computer vision tasks



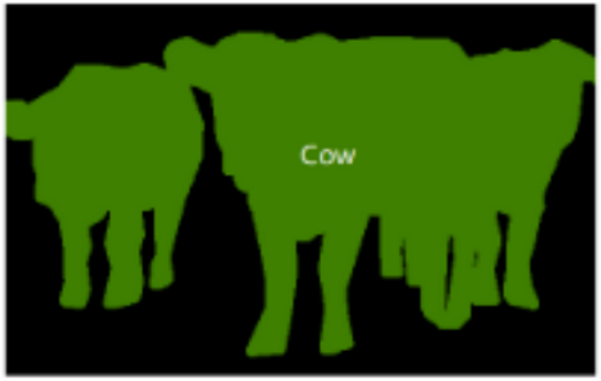
(a) Image Classification (b) Object Detection

(c) Semantic Segmentation (d) Instance Segmentation

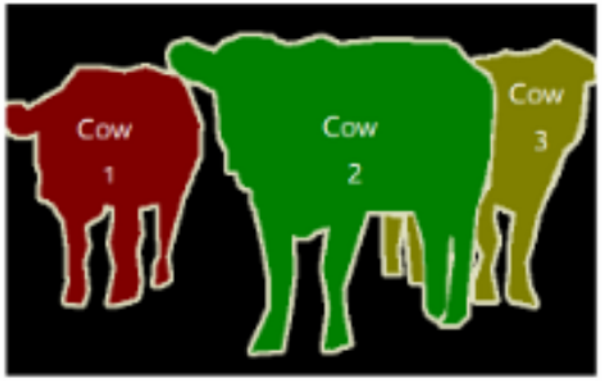
Fundamental computer vision tasks

(a) Image Classification

(b) Object Detection

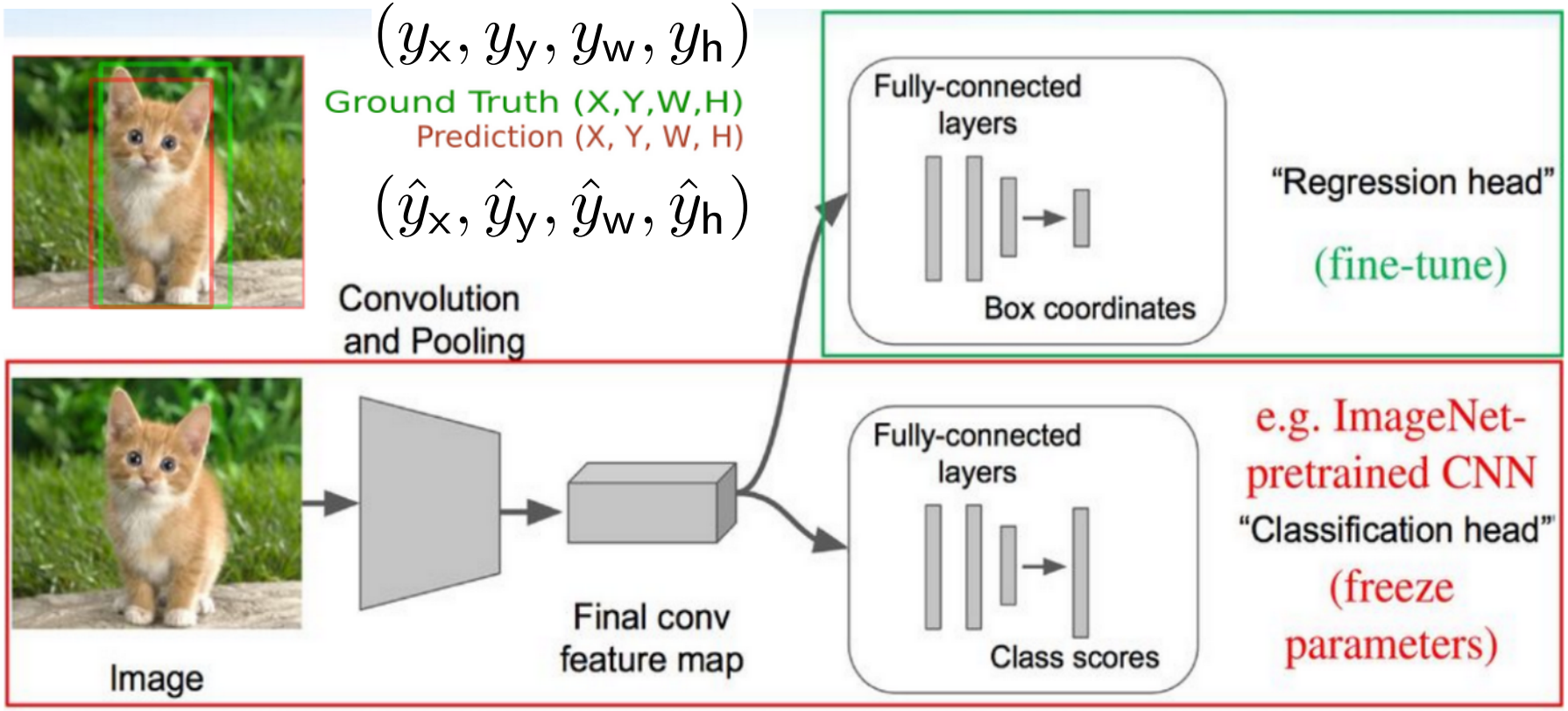


(c) Semantic Segmentation



(d) Instance Segmentation

Object localization: single object



- **Localization can be seen as regression task**

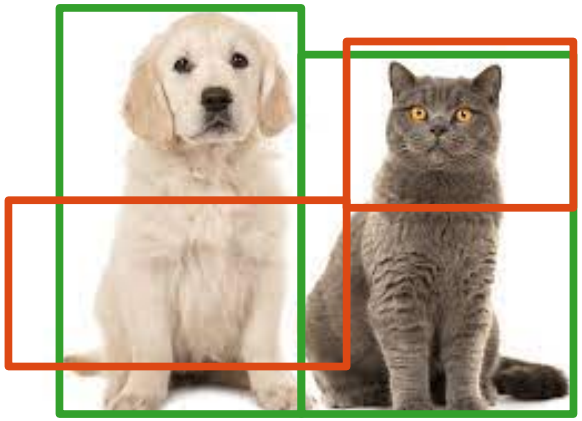
  predictions: $(\hat{y}_x, \hat{y}_y, \hat{y}_w, \hat{y}_h)$ labels: (y_x, y_y, y_w, y_h)

$(\hat{y}^i_{k,x}, \hat{y}^i_{k,y}, \hat{y}^i_{k,w}, \hat{y}^i_{k,h})$

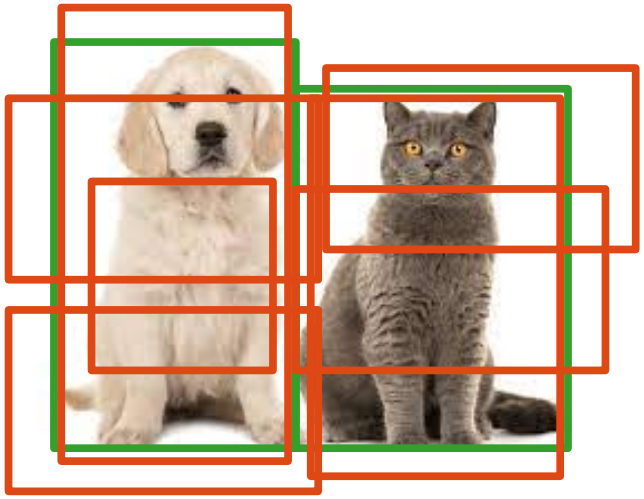
Object detection: multiple objects

Metrics

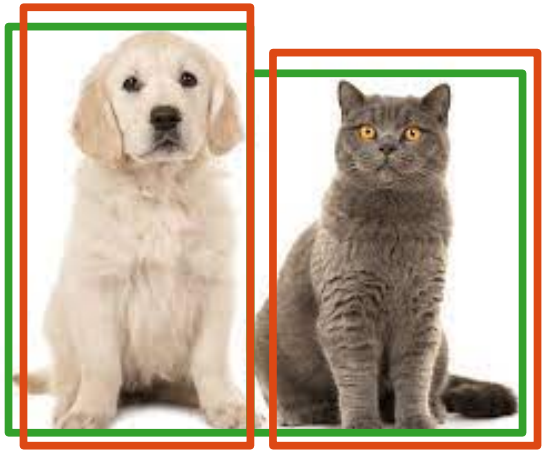
Ground truth bounding boxes



Low overlap; false negatives

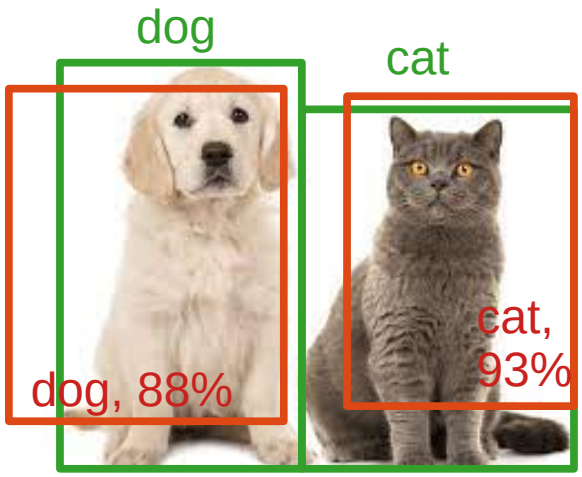


False positives



Good predictions

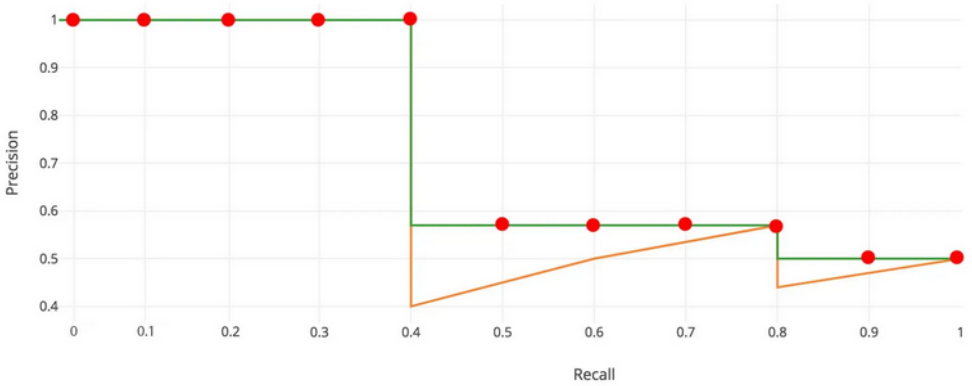
Metrics: methods provide bounding boxes, class and confidence score



Metrics: mean Average Precision (mAP)

- Run detector with varying thresholds
  - Actually: go through list of predicted objects sorted by confidence value
- Assign detections to closest object
- Count **TP**, **FP, FN**
- Compute Average Precision (AP)

True positives (TP): Number of correctly detected objects (IoU>0.5)
False negatives (FN): Number of objects not detected (IoU<0.5)
False positives FP: Wrong detections



Precision $$P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}$$

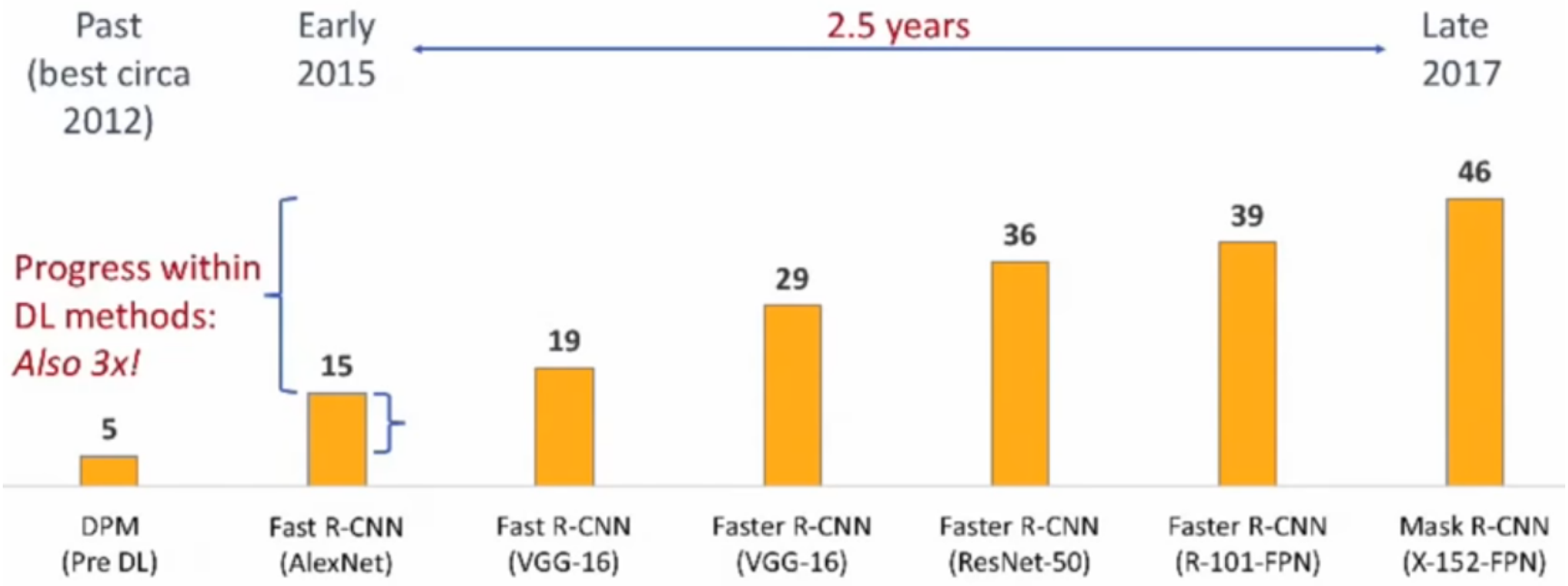
Recall $$R = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}$$

$$\mathrm{AP} = \frac{1}{11} \sum_{R \in \{0, 0.1, \ldots, 1.0\}} \max_{R' \geq R} P(R')$$

Overview of object detection approaches

- **Overfeat (2013):** exhaustive search and CNN features
- **SPPNet (2014):** spatial pyramid pooling
- **R-CNN (2014):** selective search with AlexNet
- **Fast R-CNN (2015):** ROI pooling and dual heads
- **Faster R-CNN (2015):** Region proposal networks
- **Mask-RNN (2017):** Segmentation head/branch and RoIAlign
- **YOLO (2016):** Multiple simultaneous outputs and non-maximum suppression
- **SSD (2016):** re-use of intermediate representations

Progress with Deep Learning methods



Notation

Note: Notation in this Section

The notation in this Section can become quite cluttered due to the complex loss functions that are employed for the presented architectures. Overall we have a number of possible entities that we have to represent:

- Coordinates of bounding boxes: Objects are typically located in bounding boxes that are given by their x- and y-coordinates, their width and their height. We chose x, y, w, and h for these quantities. Note that the type of font distinguishes them from the input object $\boldsymbol{x}$, the label $\boldsymbol{y}$, a weight vector $\boldsymbol{w}$ and a hidden representation $\boldsymbol{h}$.

- Network outputs and labels: Some architecture have two output layers, sometimes called *heads*, one for classifying a sub-image, and the other one for predicting bounding boxes. We will use $(\hat{y}_\mathsf{x}, \hat{y}_\mathsf{y}, \hat{y}_\mathsf{w}, \hat{y}_\mathsf{h})$ for the predictions and $(y_\mathsf{x}, y_\mathsf{y}, y_\mathsf{w}, y_\mathsf{h})$ for the labels of the regression output layer. For the classification output layer, we will use $\hat{\boldsymbol{p}} = (\hat{p}_1, \ldots, \hat{p}_k)$ and $\boldsymbol{p} = (p_1, \ldots, p_k)$ for the predictions and labels, respectively.

Notation

- Sub-images or regions: In this Section, there are often multiple sub-images of a single input image. These sub-images are called "regions", and networks often make predictions per region. We will use a superscript ^{i} to denote the predictions for region i.
- Bounding boxes: With the notation above, a bounding box Y for the i region is denoted by $\mathsf{Y}^i = (y^i_{\mathsf{x}}, y^i_{\mathsf{y}}, y^i_{\mathsf{w}}, y^i_{\mathsf{h}})$ and a predicted bounding box, a so-called proposal, is denoted as $\hat{\mathsf{Y}}^i = (\hat{y}^i_{\mathsf{x}}, \hat{y}^i_{\mathsf{y}}, \hat{y}^i_{\mathsf{w}}, \hat{y}^i_{\mathsf{h}})$
- Worst-case scenario "full notation": We aim at keeping the notation uncluttered by removing unnecessary indices. In the worst case, we can encounter a quantity such as

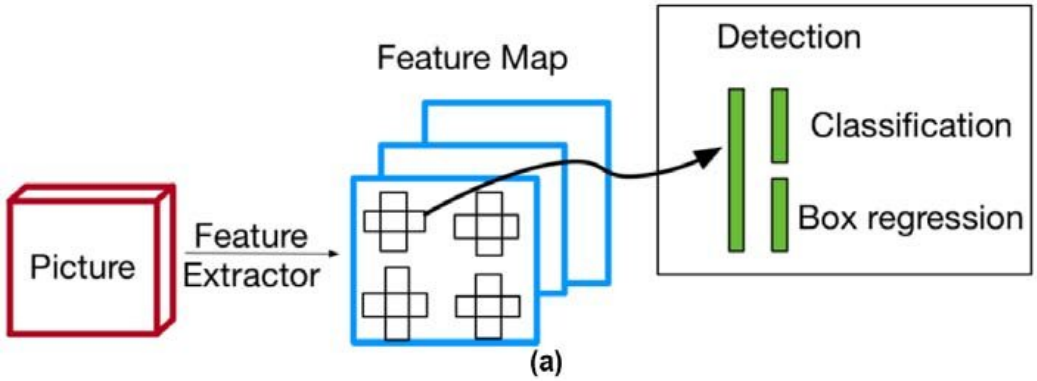
$$\hat{y}^i_{k,\mathsf{x}} \tag{2.8}$$

which is the prediction y, for class k and x-coordinate x for the i-th region of the image.

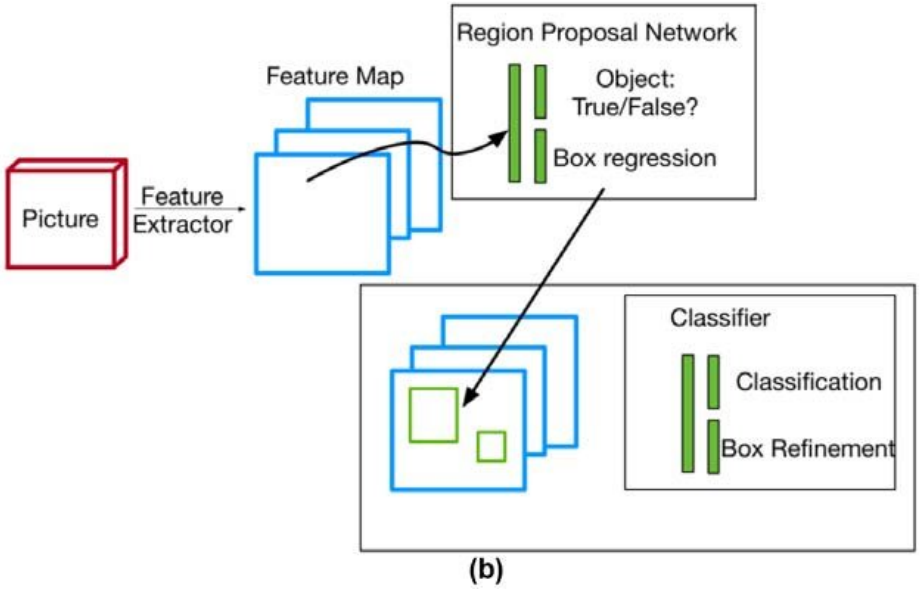
Overfeat (2013)

- AlexNet is used as backbone
- hundreds of localization predictions are retrieved by **processing the image on multiple scales** and with **multiple strides**
  - Returns image with spatial information (high-level feature maps)
  - Uses regression layer to predict BB coordinates
- greedy aggregation algorithm takes all these predicted bounding boxes and consecutively merges them
- Winner object localization ImageNet localization challenge

Object detection: one-stage vs two-stage approaches



One-stage



Two-stage

R-CNN (2014)

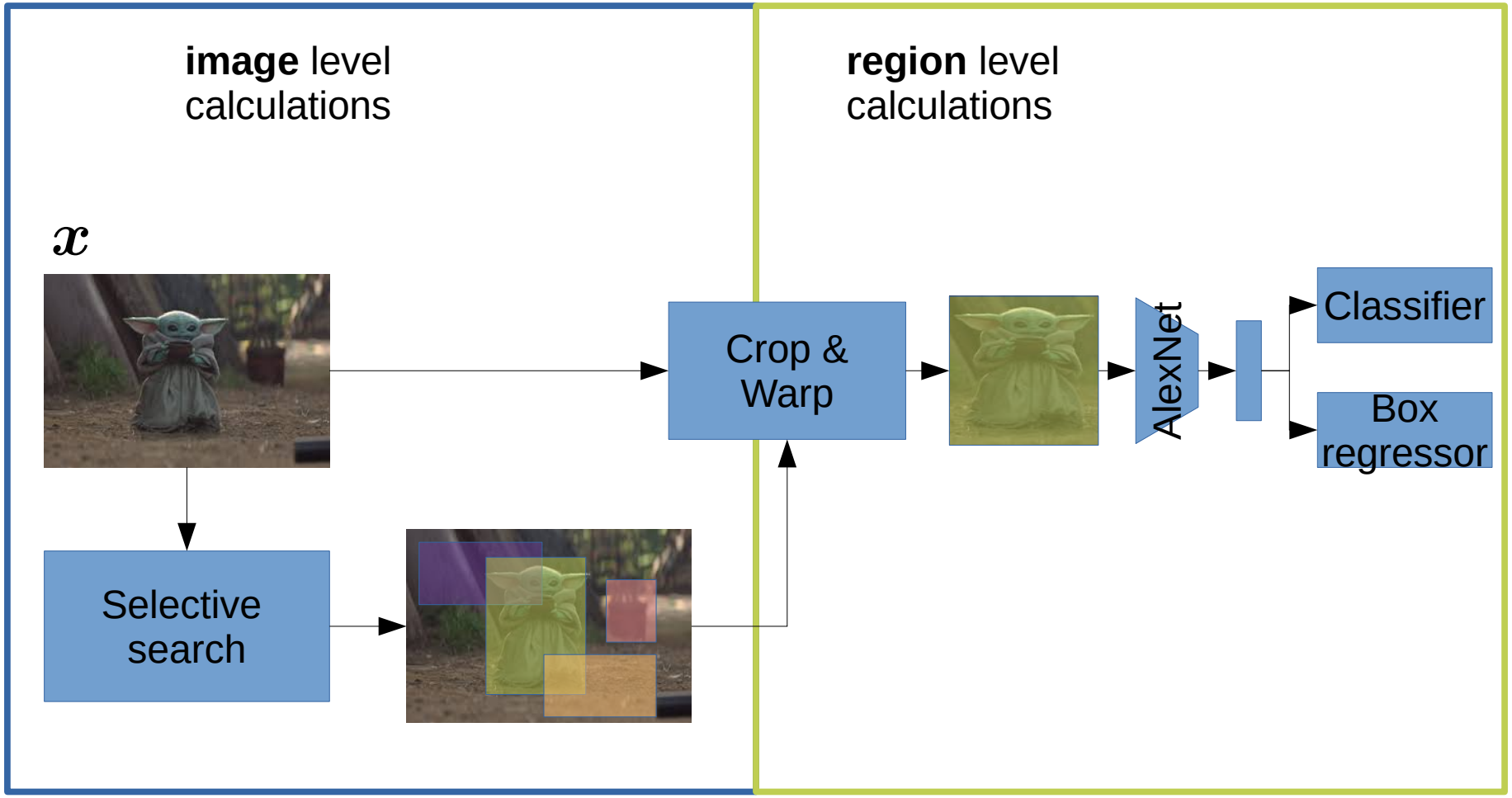
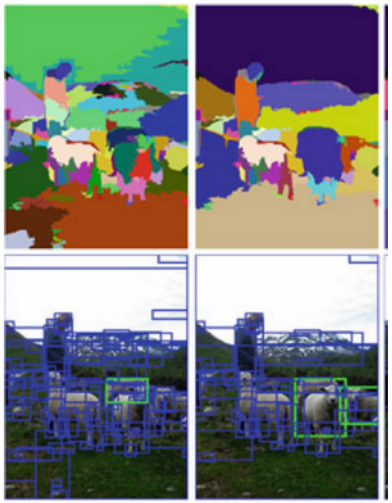


image level calculations
region level calculations
x
Selective search
Crop & Warp
AlexNet
Classifier
Box regressor

R-CNN (2014): Selective search

- Heuristic algorithm that suggests bounding boxes across objects
- “Clusters pixels” based on several criteria (e.g. similar colors)



Algorithm 1: Hierarchical Grouping Algorithm

DontPrintSemicolon **Input**: (colour) image
Output: Set of object location hypotheses L

Obtain initial regions $R = \{r_1, \cdots, r_n\}$ using Felzenszwalb and Huttenlocher (2004) Initialise similarity set $S = \emptyset$;
foreach *Neighbouring region pair* (r_i, r_j) **do**
  Calculate similarity $s(r_i, r_j)$;
  $S = S \cup s(r_i, r_j)$;

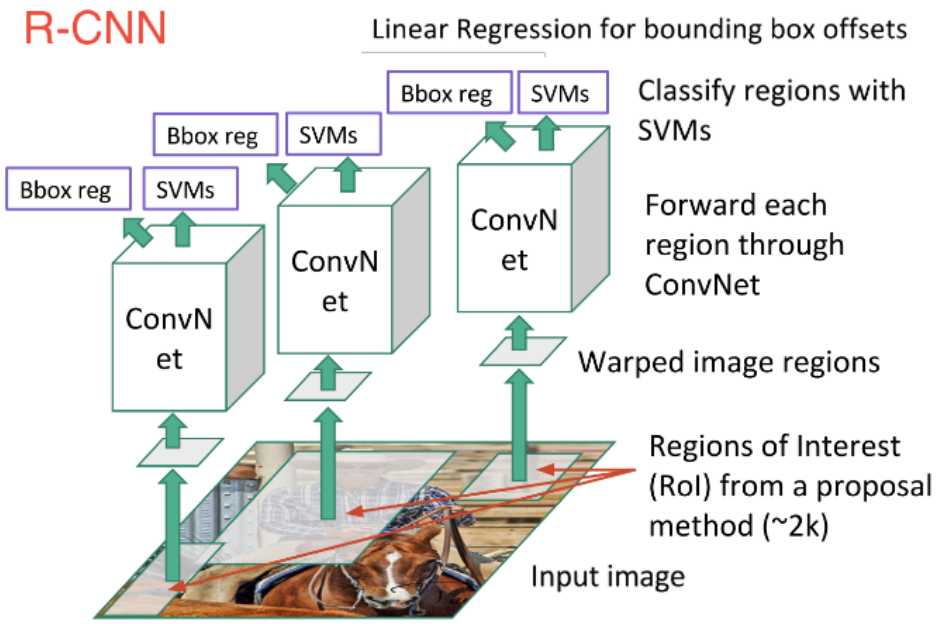
while $S \neq \emptyset$ **do**
  Get highest similarity $s(r_i, r_j) = \max(S)$;
  Merge corresponding regions $r_t = r_i \cup r_j$;
  Remove similarities regarding r_i : $S = S \setminus s(r_i, r_*)$;
  Remove similarities regarding r_j : $S = S \setminus s(r_*, r_j)$;
  Calculate similarity set S_t between r_t and its neighbours;
  $S = S \cup S_t$;
  $R = R \cup r_t$;
Extract object location boxes L from all regions in R;

R-CNN (2014): CNN transform

- Uses AlexNet to process regions of interest (ROI)
- Provides features for each ROI

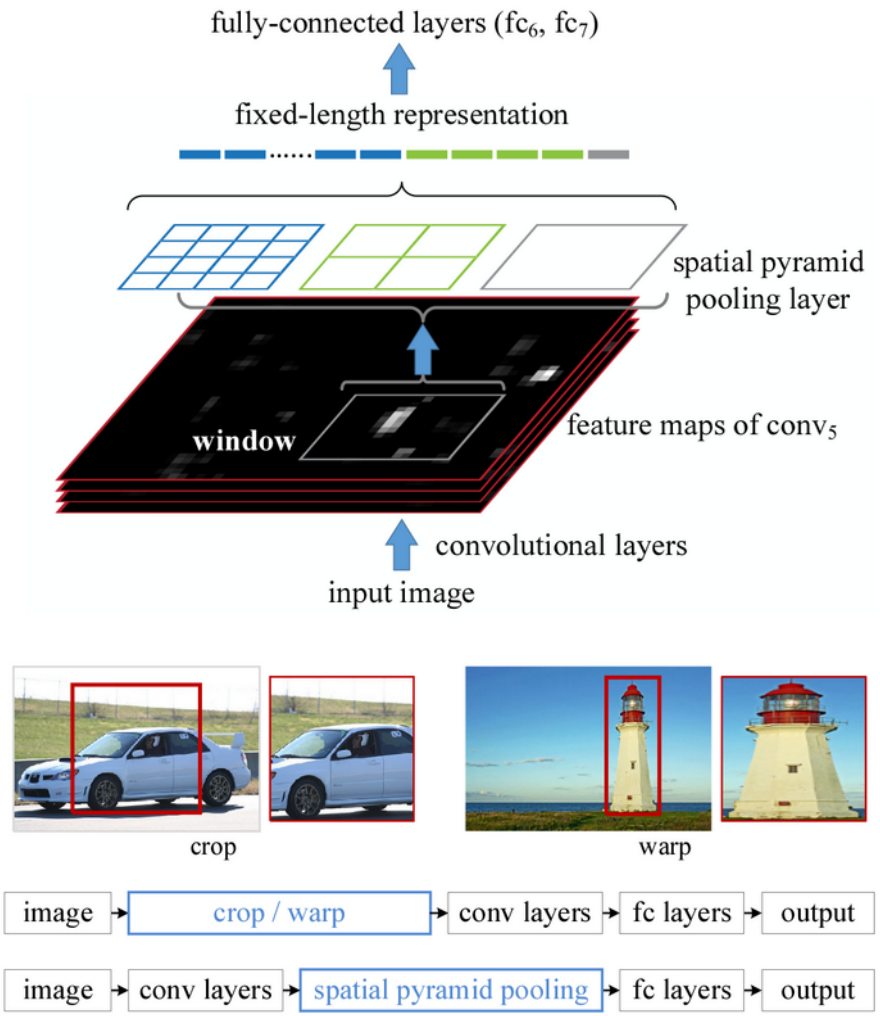
R-CNN (2014): SVM classification and regression

- SVM for classificatio
- Linear regression for bounding box offsets

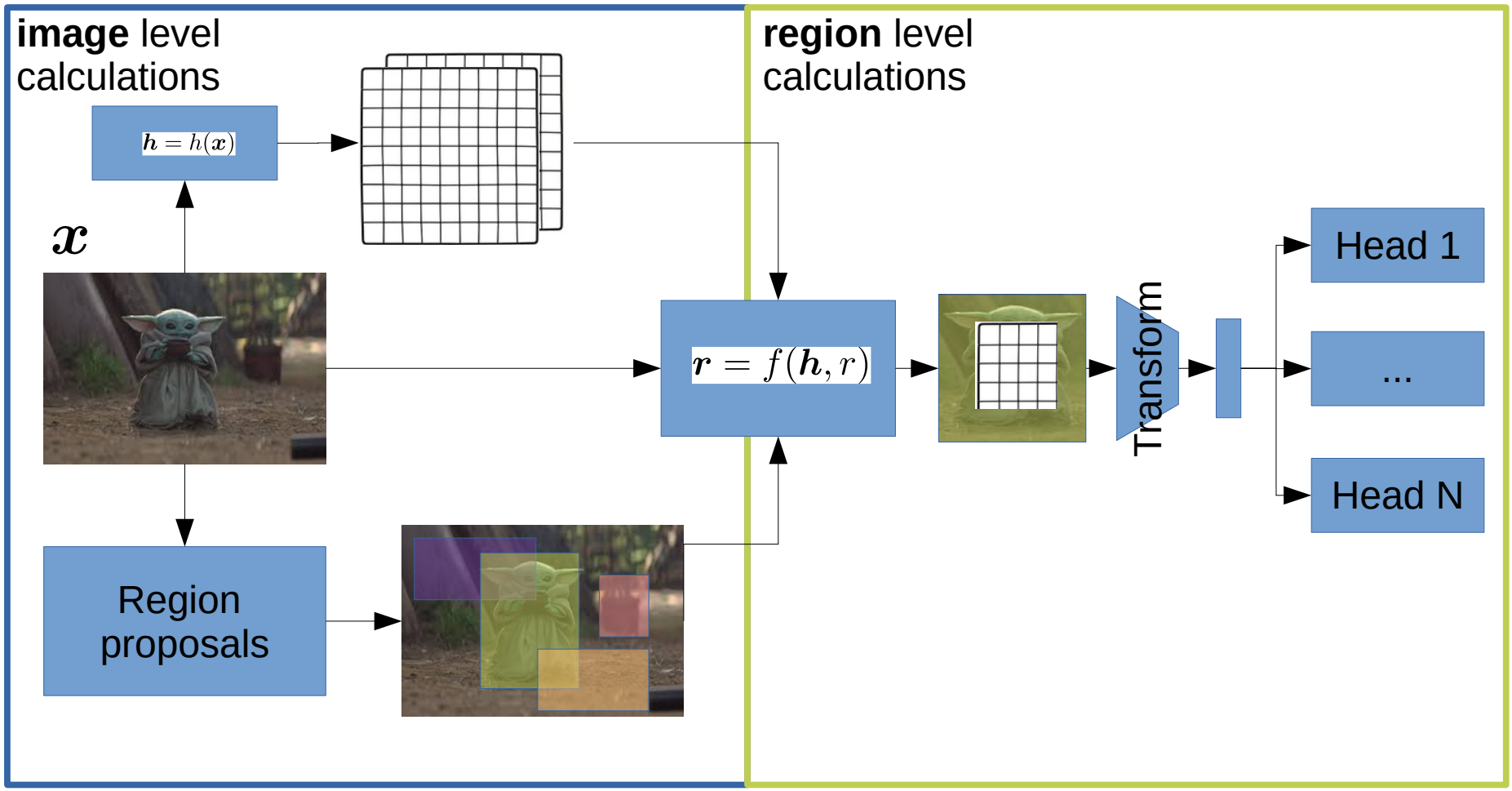


SPPNet (2014)

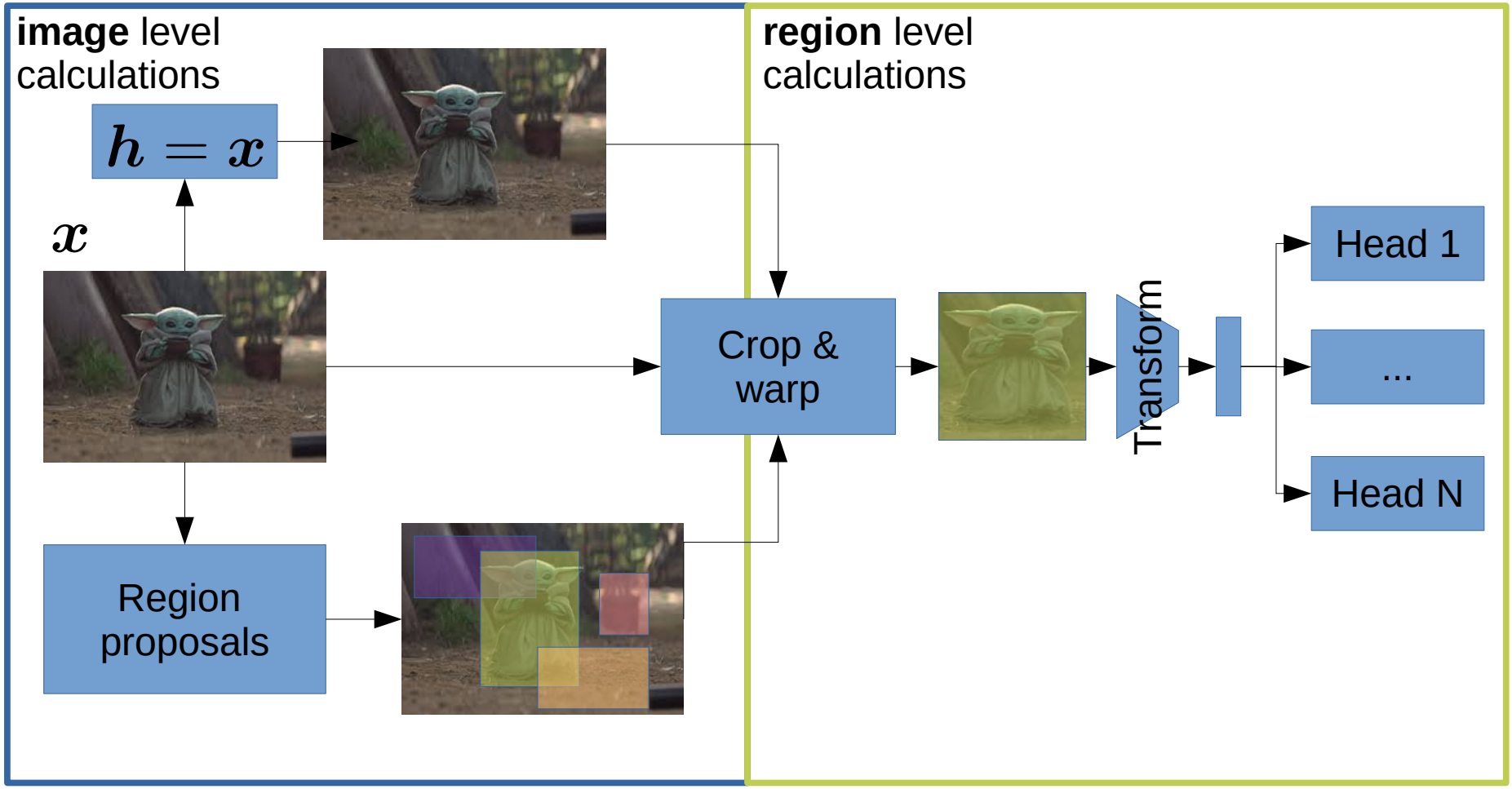
- Spatial pyramid pooling
  - Guarantees to pool inputs of variable size (height, width) into fixed-sized outputs
  - A type of pooling operation
  - Splits height and width into M equally sized bins; then pools each bin
    - Results in fixed size independent of input size
- Similar to R-CNN, but SPP instead of crop/warp



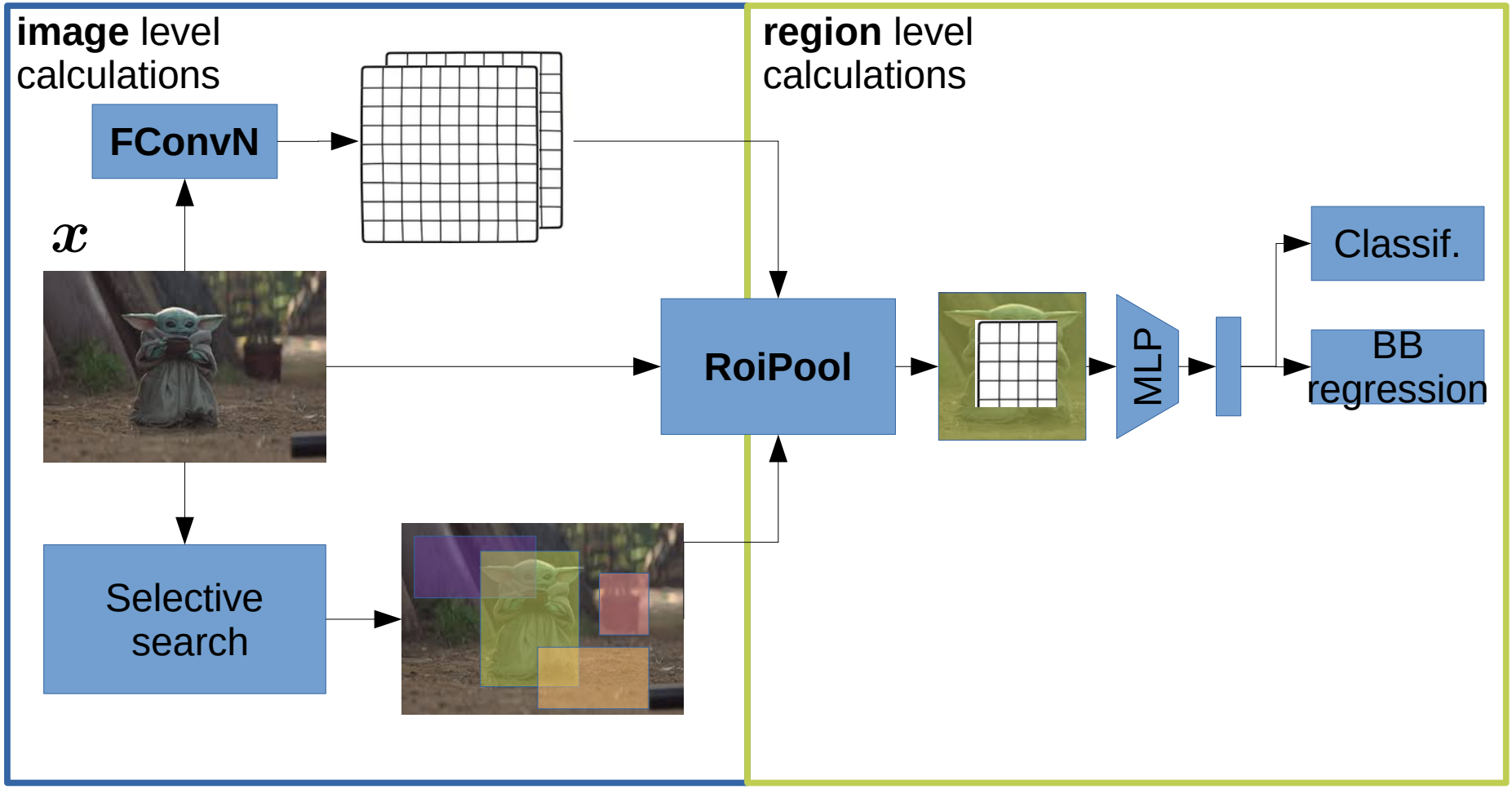
General framework



Revisiting R-CNN (2014)



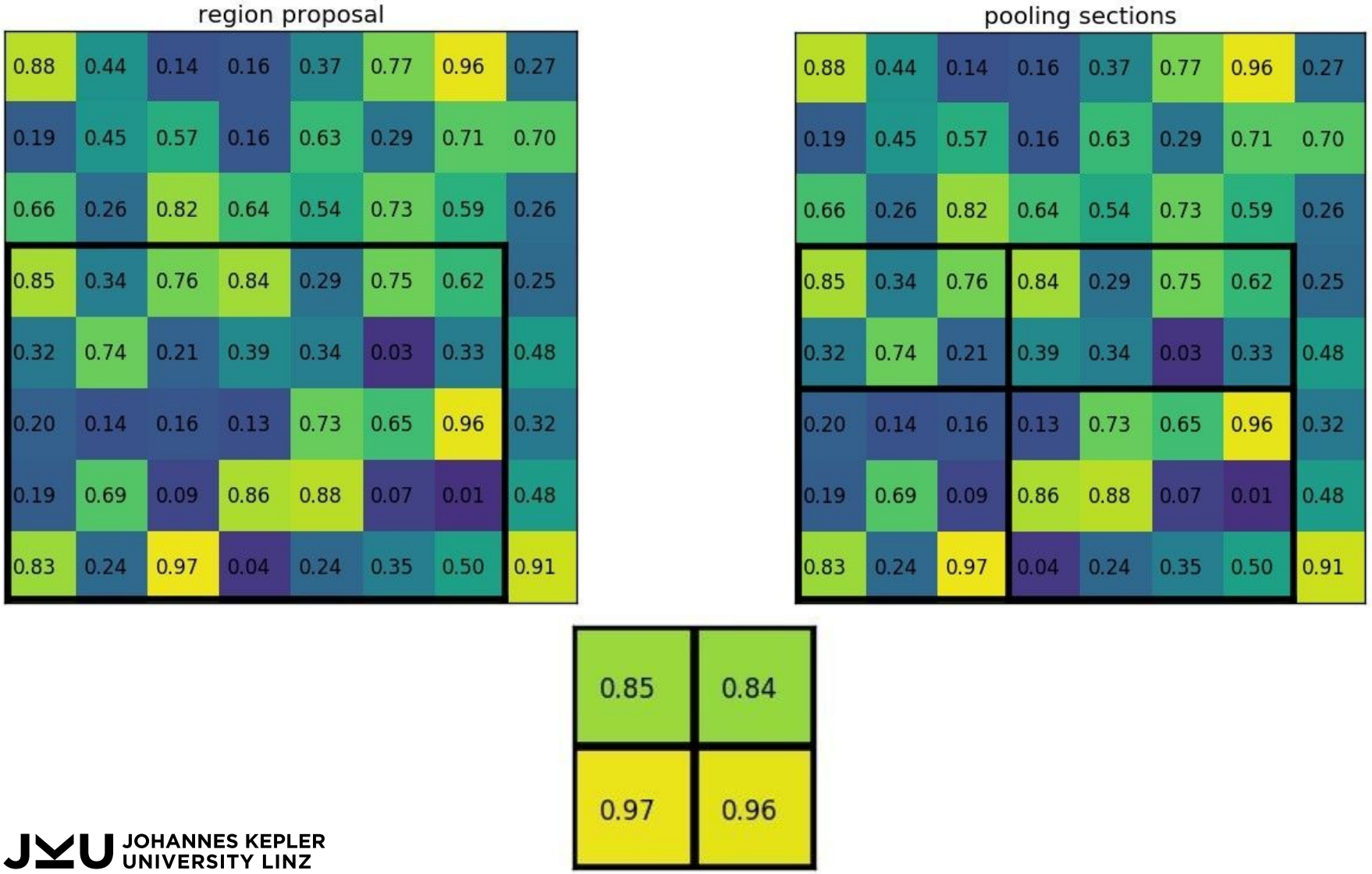
Fast R-CNN (2015)



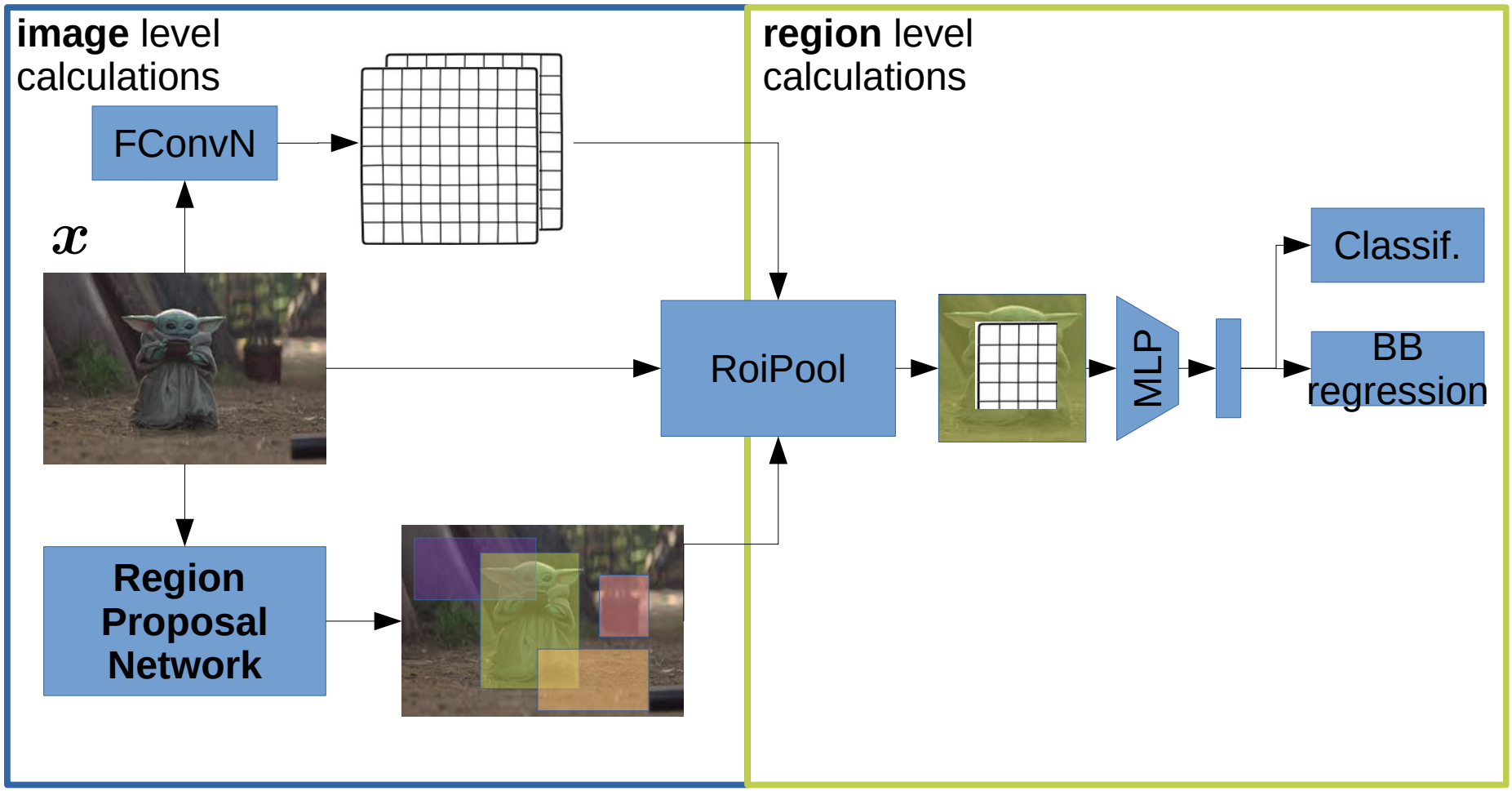
Fully-convolutional network (FCN)

- Any ConvNet can be used
  - e.g. Resnet, AlexNet, VGG, Inception
- Characterization of image with spatial information
  - Feature maps

RoI Pool: region proposal snapped to grid



Faster R-CNN (2015)



Faster R-CNN

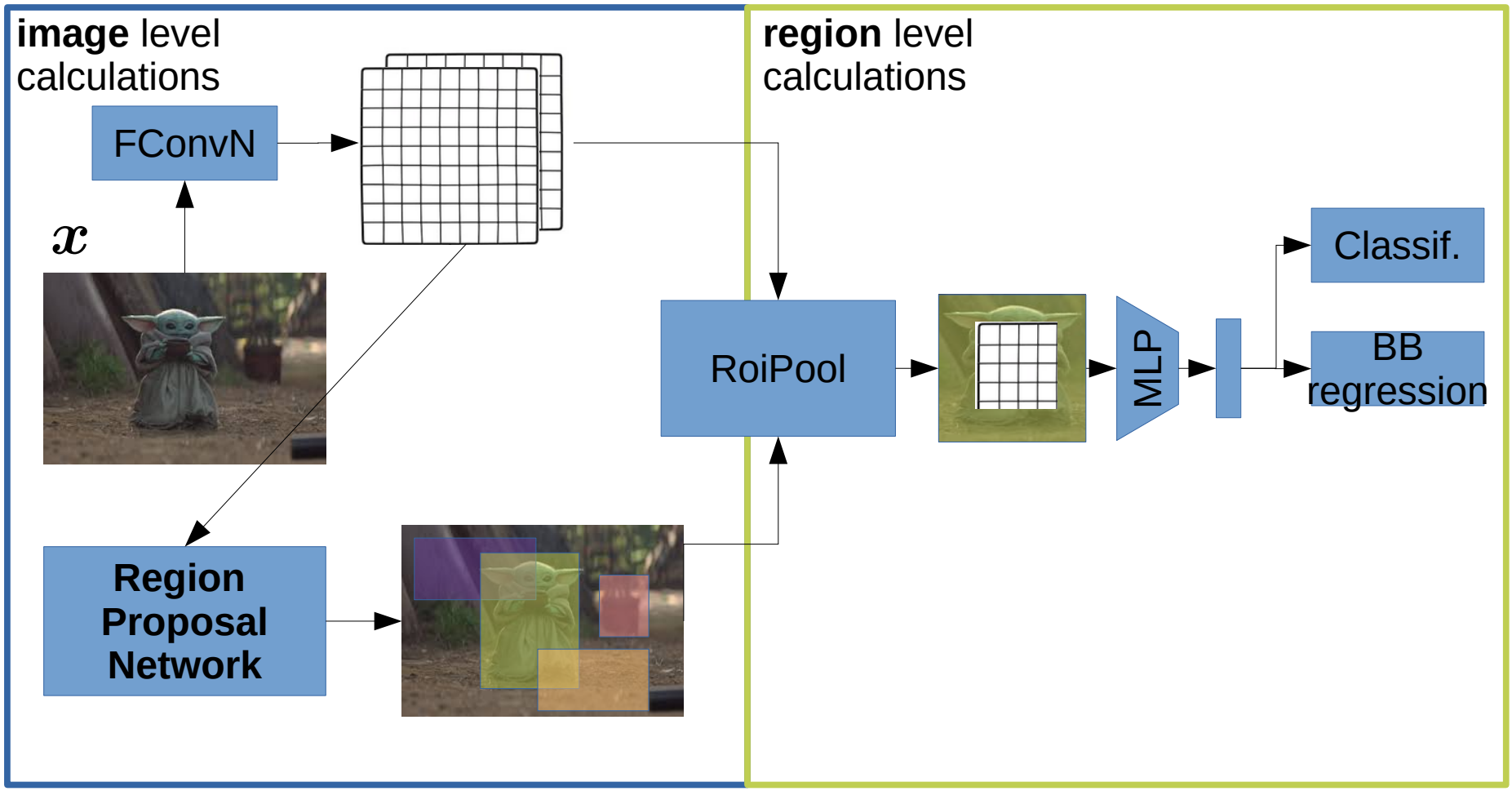
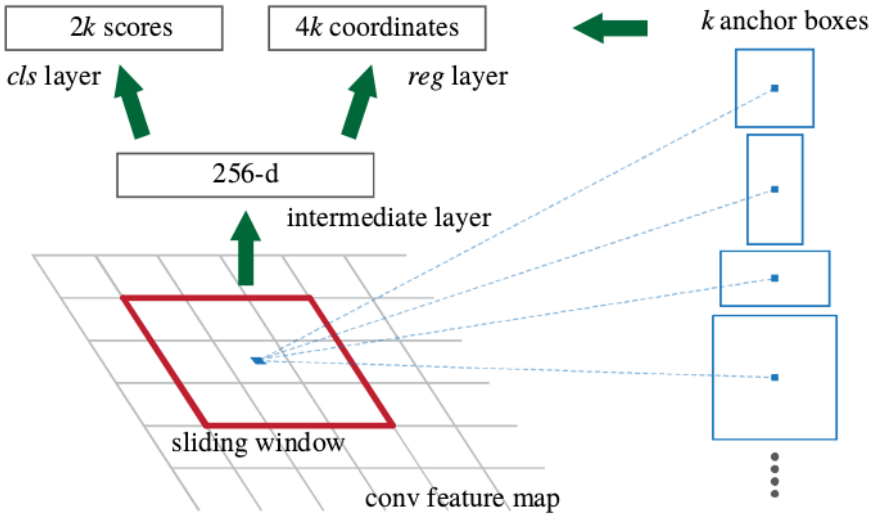


image level calculations
FConvN
x
Region Proposal Network
region level calculations
RoiPool
MLP
Classif.
BB regression

Region proposal network (RPN)

- Sliding window over feature map
- 3x3 kernel, but does not focus on 3x3 area, but on attached box (with different size and aspect ratio)
  - “looking for object”, not in 3x3, but in bigger box (which are attached to it; anchor boxes)
  - Rather tries to transform anchor boxes
    - Anchor decoupled from kernel; anchor boxes: different aspect ratios
  - Predicts transformation of anchor
  - Predicts “objectness”



Mask R-CNN

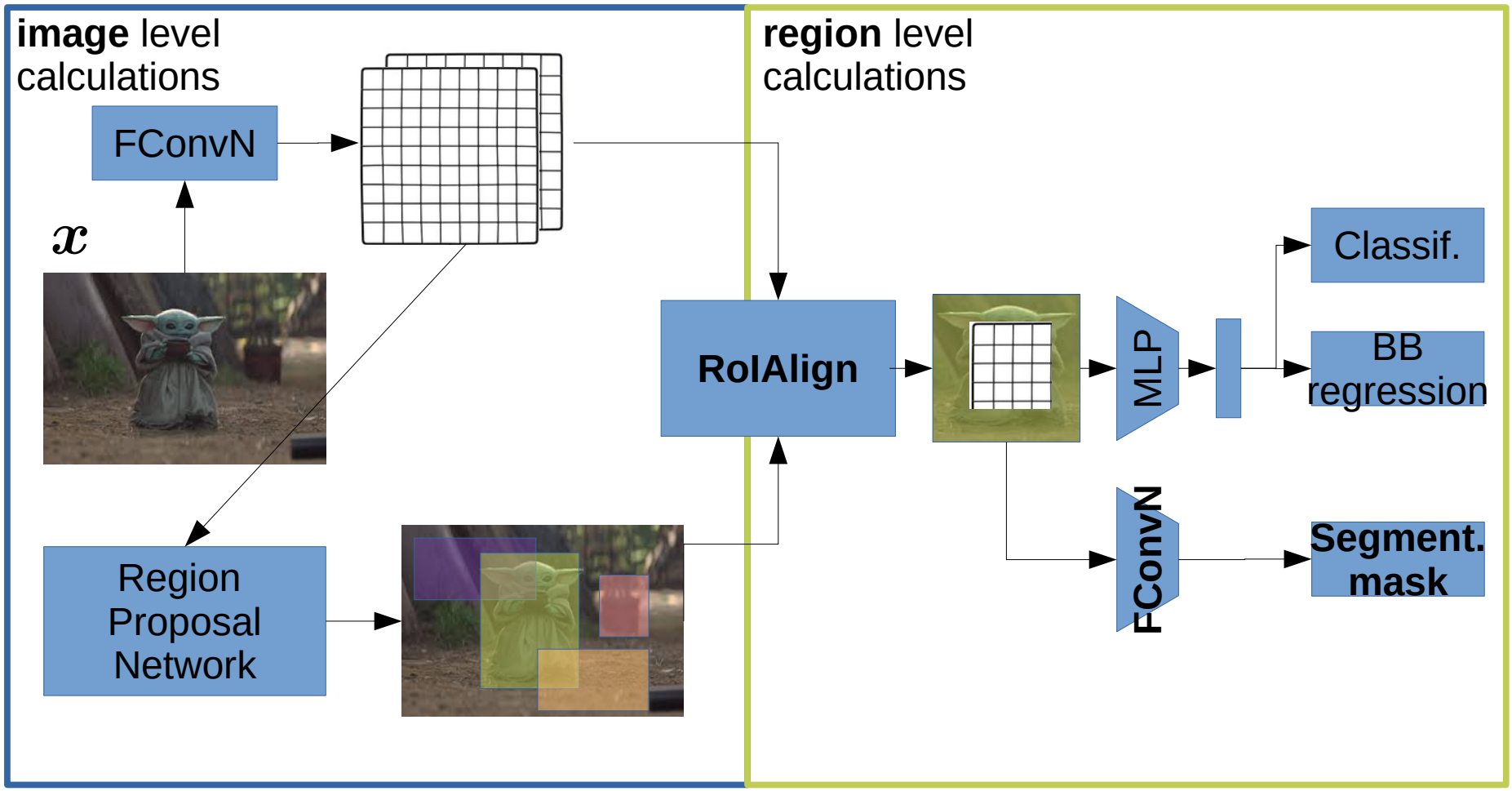
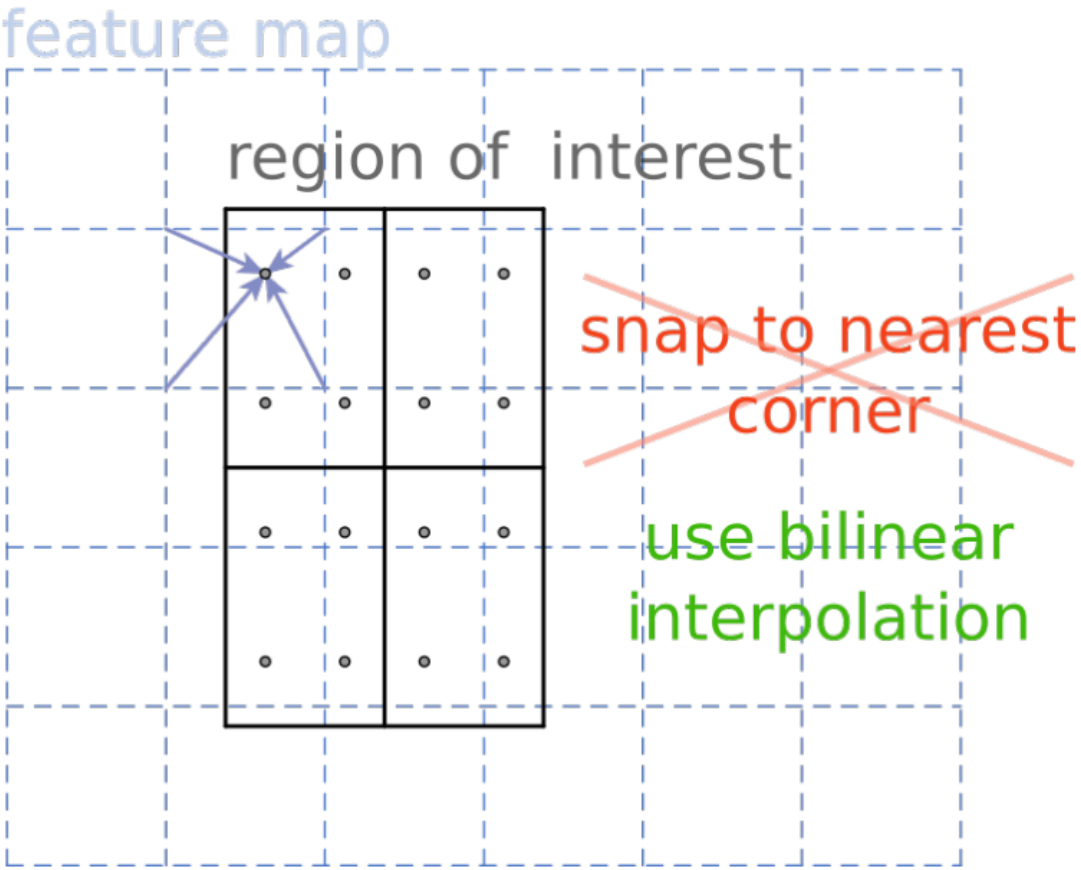
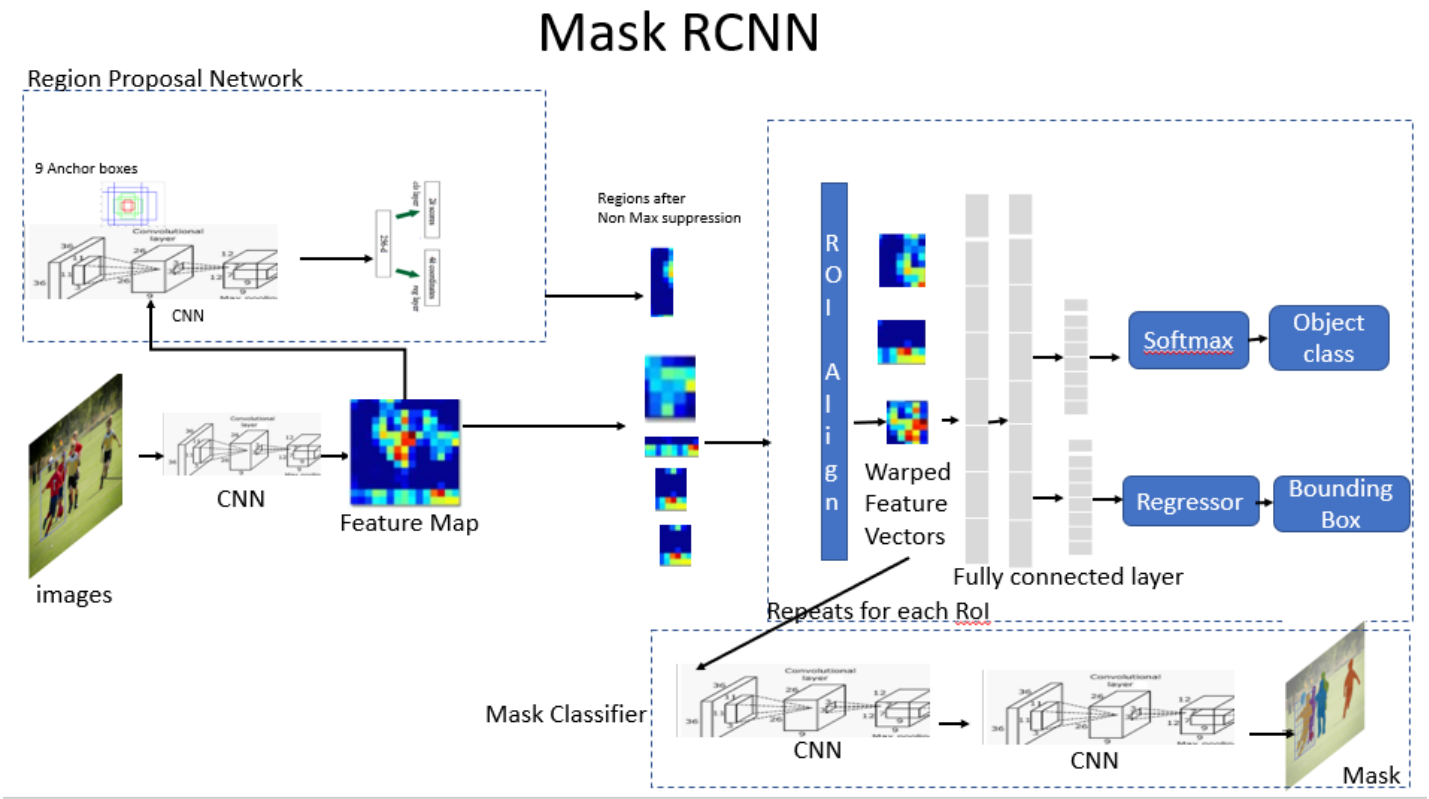


image level calculations
region level calculations
FConvN
x
Region Proposal Network
RoIAlign
MLP
Classif.
BB regression
FConvN
Segment. mask

RoI Align


feature map
region of interest
snap to nearest corner
use bilinear interpolation

Segmentation mask

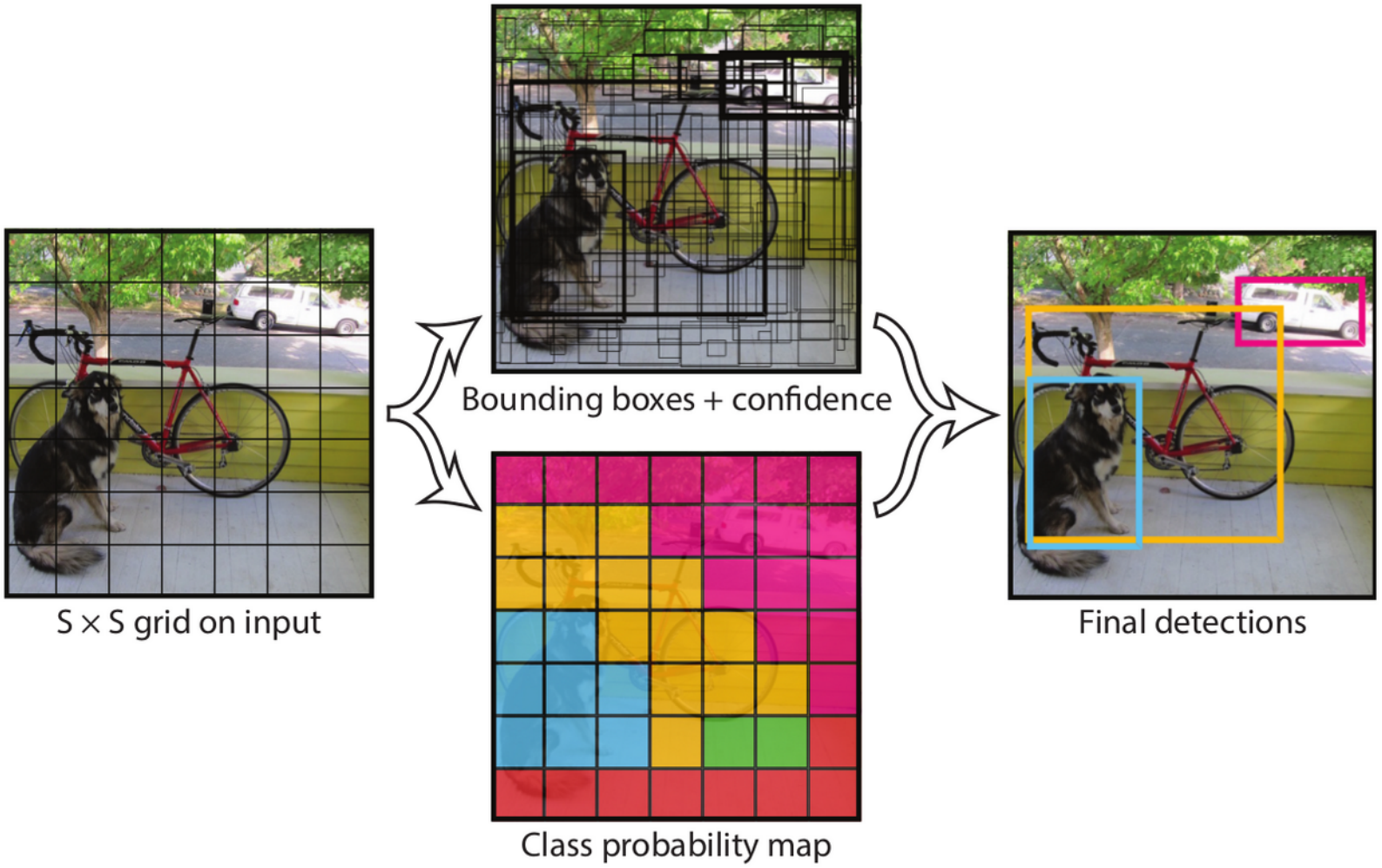


- Sigmoid activation of outputs for each class
  - No softmax (mutually exclusive classes)
- Loss is only propagated back from the segmentation mask of the known class

YOLO (2016): You only look once

- Single-stage approach
- ConvNet used to reduce image to small grid (e.g. 7x7)
- Each grid-point predicts
  - Bounding boxes
  - “objectness”
  - Classification
- Complex loss function; balancing loss terms
- Extremely fast: can be used for real-time object detection

YOLO


Bounding boxes + confidence
S × S grid on input
Class probability map
Final detections

YOLO: outputs

Network output. Each *grid cell* of the $S \times S$ grid outputs the following:

- E bounding boxes with four coordinates: $\hat{y}_{s_1,s_2,e,\mathsf{x}}$, $\hat{y}_{s_1,s_2,e,\mathsf{y}}$, $\hat{y}_{s_1,s_2,e,\mathsf{w}}$, $\hat{y}_{s_1,s_2,e,\mathsf{h}}$,
- a confidence for the predicted bounding box: $\hat{y}_{s_1,s_2,e,\mathsf{c}}$,
- and the class labels (vector of length K) of that grid cell: $\hat{y}_{s_1,s_2,k}$.

Let us assume, that we have a task, in which $K = 20$ classes have to be detected, and we use a grid size of $S = 7$, and each grid cell predicts $E = 2$ bounding boxes. Then, the size of the output vector $\boldsymbol{y}$ is: $S \times S \times (5E + K)$. Thus $\boldsymbol{y} \in \mathbb{R}^{7\times7\times30}$.

YOLO: loss function

$$L(\boldsymbol{y}, \hat{\boldsymbol{y}}) = \lambda_1 \sum_{s_1,s_2}^{S-1} \sum_{e=0}^{E-1} \mathbb{1}^{\text{obj}}_{s_1,s_2} \left((\hat{y}_{s_1,s_2,e,\mathsf{x}} - y_{s_1,s_2,e,\mathsf{x}})^2 + (\hat{y}_{s_1,s_2,e,\mathsf{y}} - y_{s_1,s_2,e,\mathsf{y}})^2 \right) \tag{2.14}$$

$$+ \lambda_2 \sum_{s_1,s_2}^{S-1} \sum_{e=0}^{E-1} \mathbb{1}^{\text{obj}}_{s_1,s_2} \left((\sqrt{\hat{y}_{s_1,s_2,e,\mathsf{w}}} - \sqrt{y_{s_1,s_2,e,\mathsf{w}}})^2 + (\sqrt{\hat{y}_{s_1,s_2,e,\mathsf{h}}} - \sqrt{y_{s_1,s_2,e,\mathsf{h}}})^2 \right) \tag{2.15}$$

$$+ \lambda_3 \sum_{s_1,s_2}^{S-1} \sum_{e=0}^{E-1} \mathbb{1}^{\text{obj}}_{s_1,s_2} (\hat{y}_{s_1,s_2,e,\mathsf{c}} - y_{s_1,s_2,e,\mathsf{c}})^2 \tag{2.16}$$

$$+ \lambda_4 \sum_{s_1,s_2}^{S-1} \sum_{e=0}^{E-1} \mathbb{1}^{\text{no obj}}_{s_1,s_2} (\hat{y}_{s_1,s_2,e,\mathsf{c}} - y_{s_1,s_2,e,\mathsf{c}})^2 \tag{2.17}$$

$$+ \lambda_5 \sum_{s_1,s_2}^{S-1} \mathbb{1}^{\text{obj}}_{s_1,s_2} \sum_{k=1}^{K} (\hat{y}_{s_1,s_2,k} - y_{s_1,s_2,k})^2 , \tag{2.18}$$

Final output: Non-maximum suppression

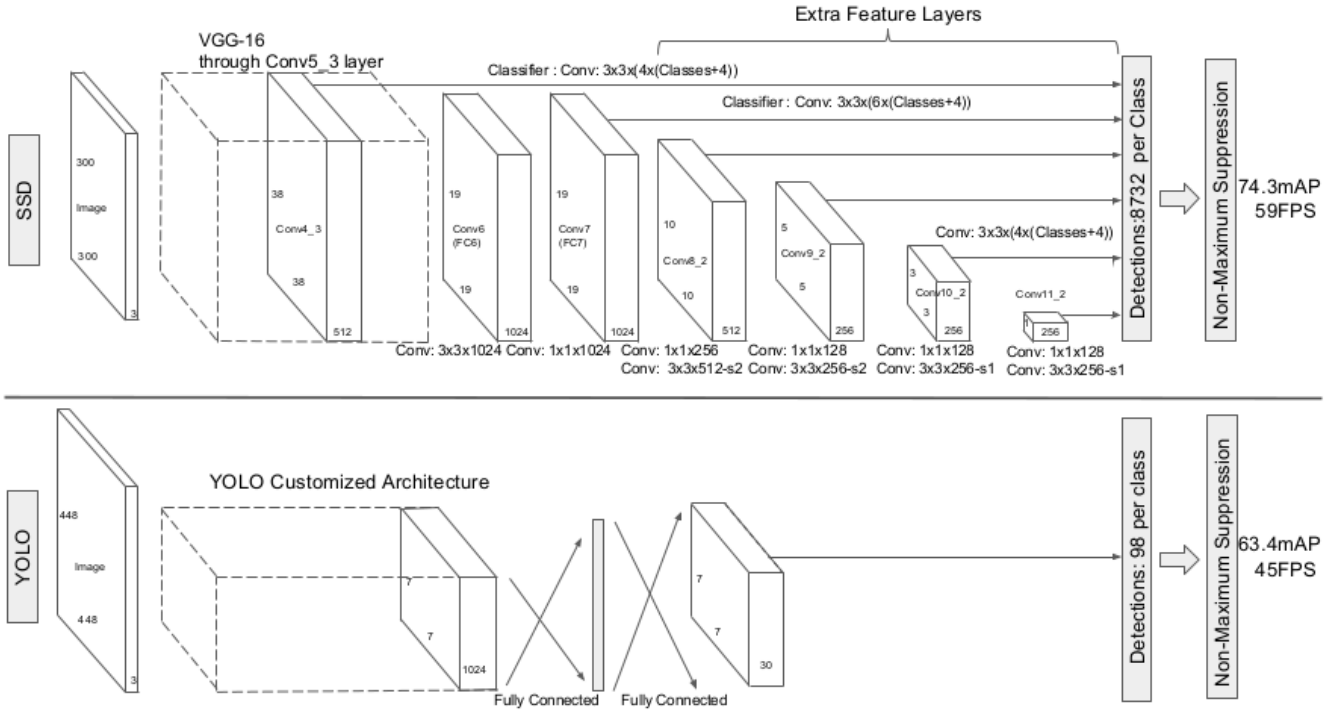
- Main idea: from many similar/overlapping bounding boxes, output only the one with highest confidence
  - IoU between predicted BB is calculated
  - BB with similar IoU removed

Final output: Non-maximum suppression

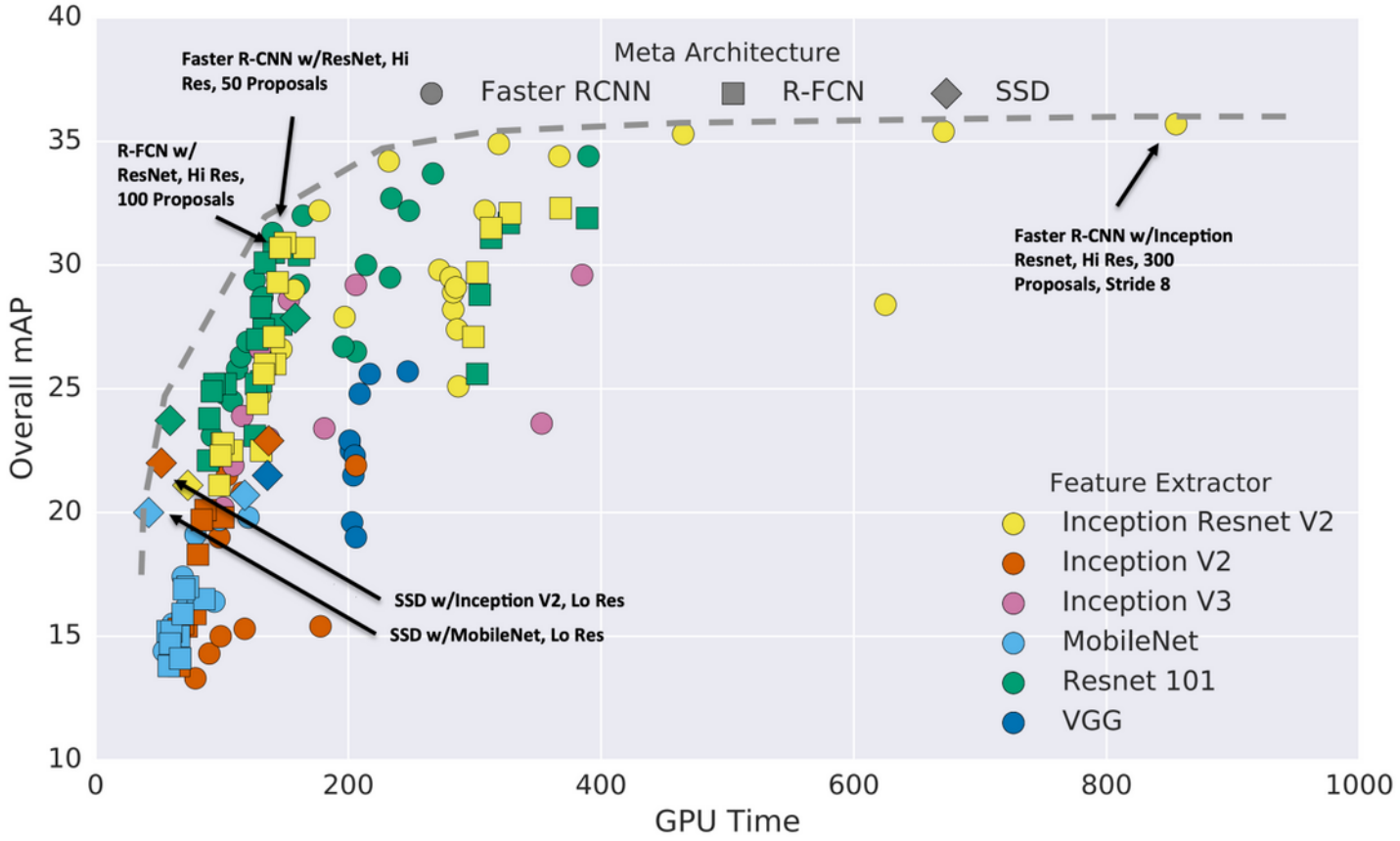
Figure 2.79: Nonmaximum Suppression takes all bounding box proposals above the confidence threshold as an input (top) and generates an output without (ideally) redundant detections (bottom). [source Hosang et al. (2017)]

Single Shot MultiBox Detector (SSD) (Liu et al., 2016)

- Single-stage approach similar to YOLO
- Uses feature maps from different scales to predict bounding boxes



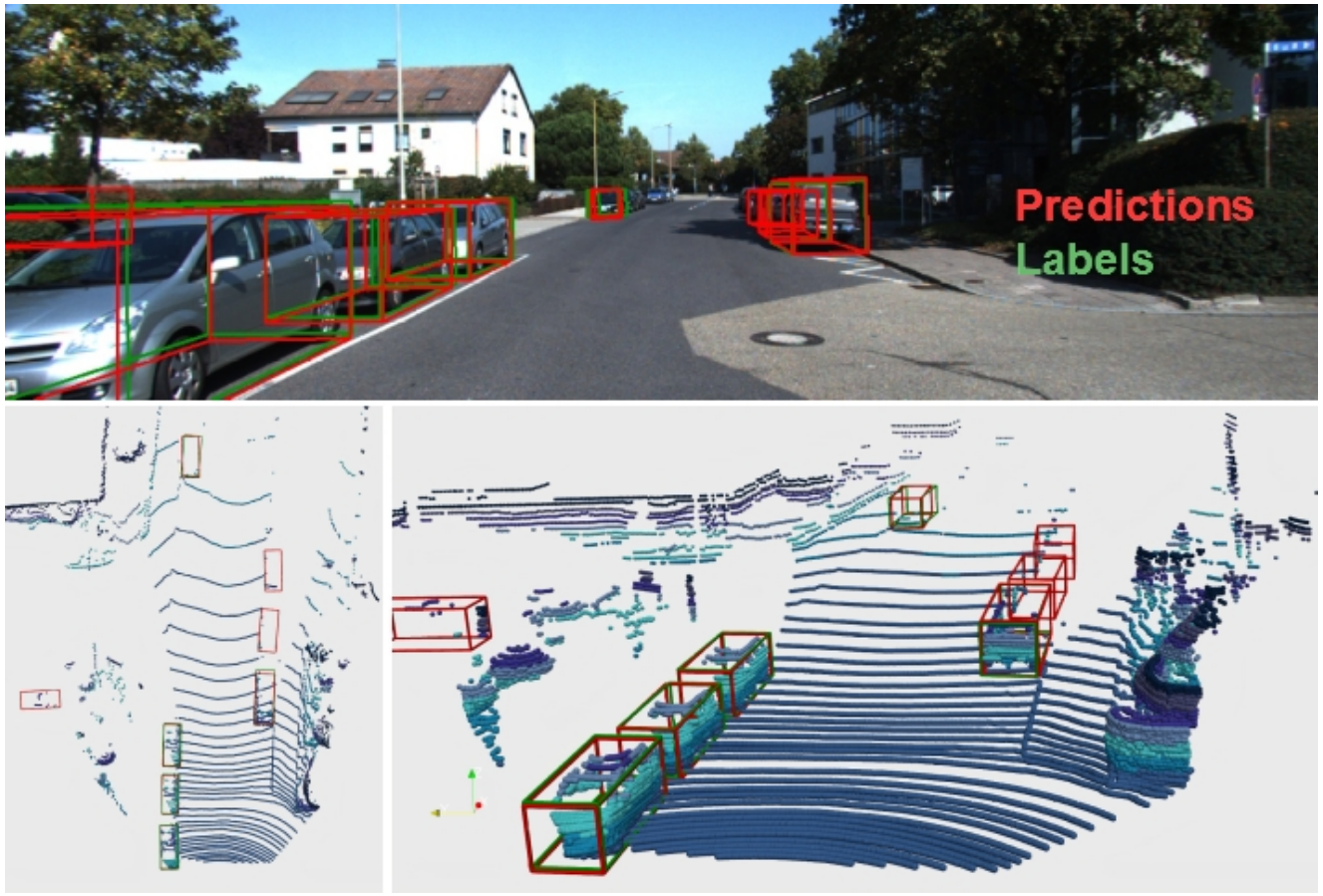
How do these methods compare?



- Two-stage methods: accurate, but slow;
- Single-stage methods: fast, but inaccurate
- Bigger backbones improve performance
- Diminishing returns for slower methods

Huang, J., Rathod, V., Sun, C., Zhu, M., Korattikara, A., Fathi, A., ... & Murphy, K. (2017). Speed/accuracy trade-offs for modern convolutional object detectors. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 7310-7311).

Research at JKU (IML & LIT AI Lab)
3D object detection; LIDAR data

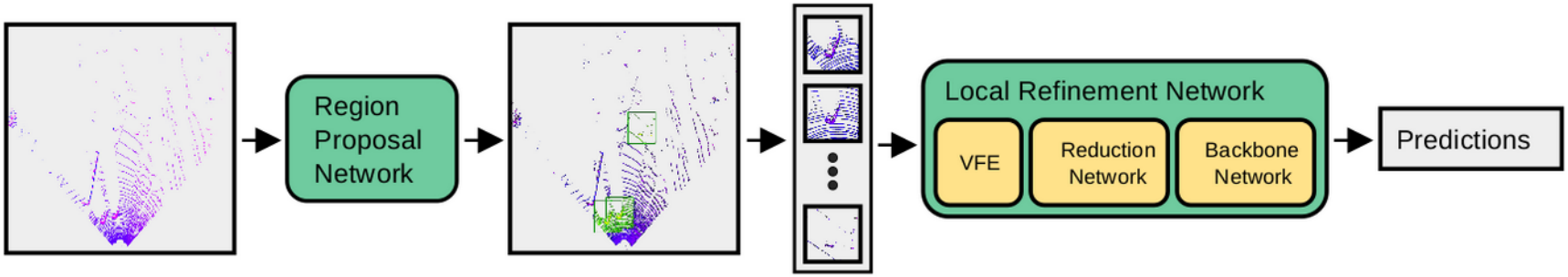


Research at JKU (IML & LIT AI Lab)
3D object detection; LIDAR data

Task: Classify each object instance and localize it with a 3D bounding box.

Application: 3D object detection is a core task in autonomous driving and robotics.

Challenge: The difficulty to localize objects is considerably harder than in 2D detection. It requires amodal, oriented bounding box prediction in a 3D space. Currently, LiDAR information is a requirement to achieve high localization accuracy. LiDAR data is represented as an unordered set of points that hold continuous coordinate values.



What happened after Mask R-CNN (2017)

- Object detection in point clouds
  - Point R-CNN, Fast Point R-CNN
- Object detection in videos
- Improvements with respect to computational efficiency

- ViT backbone with Mask R-CNN

Exploring Plain Vision Transformer Backbones for Object Detection

Yanghao Li Hanzi Mao Ross Girshick† Kaiming He†
†equal contribution

Facebook AI Research

What was not discussed in detail

- Region-based, fully-convolutional networks (Dai, 2016)
- Feature pyramid networks (Lin, 2017)

Dai, J., Li, Y., He, K., & Sun, J. (2016). R-fcn: Object detection via region-based fully convolutional networks. Advances in neural information processing systems, 29.

Lin, T. Y., Dollár, P., Girshick, R., He, K., Hariharan, B., & Belongie, S. (2017). Feature pyramid networks for object detection. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 2117-2125).

Overview of object detection approaches

- **Overfeat (2013):** exhaustive search and CNN features
- **SPPNet (2014):** spatial pyramid pooling
- **R-CNN (2014):** selective search with AlexNet
- **Fast R-CNN (2015):** ROI pooling and dual heads
- **Faster R-CNN (2015):** Region proposal networks
- **Mask-RNN (2017):** Segmentation head/branch and RoIAlign
- **YOLO (2016):** Multiple simultaneous outputs and non-maximum suppression
- **SSD (2016):** re-use of intermediate representations

Conclusions

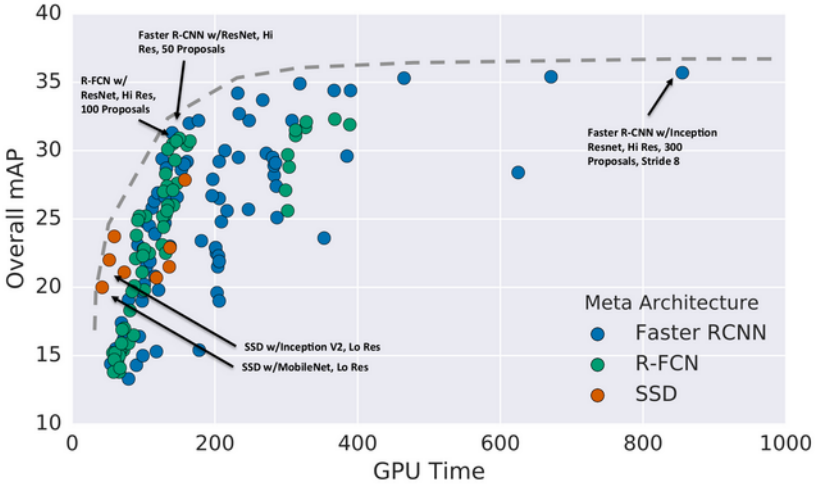
- Object detection is a difficult task due to different scales, high computational cost, the needto process sub regions of an image hundreds of times.
- IoU as main metric employed. IoU also relevant during learning, where heuristics are employed, e.g. thresholds on IoU values.
- An object can occur at multiple scales in the images.
- Balancing of object and non-object examples and balancing of loss terms is required for those architectures.
- One-stage and two-stage approaches are employed, where the former are faster and the latter are more accurate.

Summary

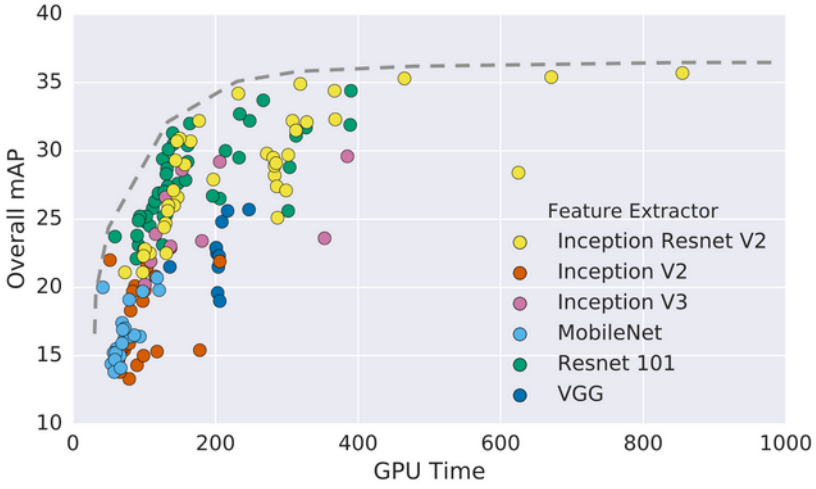
- Presented approaches for object localization and object detection
- One-stage (fast!) and two-stage approaches (accurate)
- Generalized framework for two-stage approaches
  - R-CNN, Fast R-CNN, Faster R-CNN, Mask R-CNN
- One-stage approaches:
  - YOLO, SSD

Clarifications

- In Faster R-CNN, are the two networks (RPN; classification heads) trained together with the same learning rate?
  - Suggest both “alternate training” and “joint training”
  - “4-step alternating training”: pre-train both networks, then jointly train; learning with SGD with LR 0.003 then dropped
- In the method comparison study, what are the obvious re-runs (same meta-architecture, same feature extractors)?
  - Different hyperparameters, predominantly “image size”



(a)



(b)